

AI ASSISTED CODING – ASSIGNMENT 13: REFACTORING |

Date: 05-10-2026

Name

Gottiparthi Saikiran

Enrollment No.

2603A52L01

TASK 1 | IMPROVE READABILITY

Task 1 – Improve Readability

1. Original Code

```
a = 80
b = 70
c = 90

x = (a + b + c) / 3

if x >= 50:
    print("pass")
else:
    print("fail")
```

2. AI Prompt

Suggest meaningful variable names for a, b, c, and x in the following Python code.

Do not modify the code or logic. Only suggest the meaningful variable names and explain why each name is appropriate.

Code:

```
a = 80
b = 70
c = 90

x = (a + b + c) / 3

if x >= 50:
    print("pass")
else:
    print("fail")
```

3. AI Suggestion

Copilot suggested:

- a → score_1 – the first score.
- b → score_2 – the second score.
- c → score_3 – the third score.
- x → average_score – the average of the three scores.

Copilot created a reusable calculate_average(score_1, score_2, score_3) function and kept the pass/fail condition unchanged.

For formatting and readability, Copilot added the docstring:

```
"""Return the arithmetic mean of three scores."""
This describes the purpose of the function.
```

4. Refactored Code

```
score_1 = 80
score_2 = 70
score_3 = 90

def calculate_average(score_1, score_2, score_3):
    """Return the arithmetic mean of three scores."""
    return (score_1 + score_2 + score_3) / 3

average_score = calculate_average(score_1, score_2, score_3)

if average_score >= 50:
    print("pass")
else:
    print("fail")
```

5. Test Output

Original version:

pass

Refactored version:

pass

Both versions produced the same result.

6. Explanation

The unclear variable names a, b, c, and x were replaced with meaningful names score_1, score_2, score_3, and average_score. The average calculation was moved into the reusable calculate_average() function. Formatting was improved and a docstring was added to explain the function. The original pass/fail logic was preserved. Both versions were executed and produced the same output, pass.

TASK 2 | REMOVE REPEATED CODE

Task 2 – Remove Repeated Code

1. Original Code

```
name1 = "Arun"
marks1 = 85

if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72

if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

2. AI Prompt

Look at the following Python code and identify the duplicated logic.

Do not modify the code yet.

Only identify what logic is repeated and explain why it is considered duplicated.

Code:

```
name1 = "Arun"
marks1 = 85

if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72

if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

3. AI Suggestion

The duplicated logic is the pass/fail check and output. The same if marks >= 50 condition and corresponding Pass/Fail output.

Copilot suggested refactoring this repeated logic into a reusable function named `print_result(name, marks)`.

4. Refactored Code

```
def print_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
        print(name, "Fail")

name1 = "Arun"
marks1 = 85
print_result(name1, marks1)

name2 = "Priya"
marks2 = 72
print_result(name2, marks2)

name3 = "Rahul"
marks3 = 45
print_result(name3, marks3)

name4 = "Meena"
marks4 = 90
print_result(name4, marks4)
```

5. Test Output

Original version:
Arun Pass
Priya Pass

Refactored version:
Arun Pass

```
Priya Pass  
Rahul Fail  
Meena Pass
```

The original students, Arun and Priya, produced the same results after refactoring. Rahul and Meena were additionally tested as required.

6. Explanation

The pass/fail rule now lives in one place inside `print_result()`. If the passing mark changes, only the function needs to be updated, and all students will use the updated rule. This avoids having multiple copies of the same logic that could accidentally become inconsistent. Removing duplication therefore improves maintainability.

TASK 3 | REFACTOR A LONG CONDITIONAL

Task 3 – Refactor a Long Conditional

1. Original Code

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

2. AI Prompt

Look at the following Python code and simplify the conditional logic. Do not create a function yet. Do not change behavior. Only suggest how logic can be simplified and explain why equivalent.

3. AI Suggestion

The repeated Medium Priority conditions can be combined using an OR condition:

```
if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 or temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

The first condition handles cases where both thresholds are met. If that condition fails, the OR condition handles cases where either threshold is met.

4. Refactored Code

```
age = 68
temperature = 39

def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"

print("Test 1:", determine_priority(68, 39))
print("Test 2:", determine_priority(68, 37))
print("Test 3:", determine_priority(40, 39))
print("Test 4:", determine_priority(40, 37))
```

5. Test Output

Test 1: High Priority

Test 2: Medium Priority

Test 3: Medium Priority

Test 4: Normal Priority

6. Explanation

The original and refactored logic produce the same results for all four tested age and temperature combinations:

- Age 68, Temperature 39 → High Priority
- Age 68, Temperature 37 → Medium Priority
- Age 40, Temperature 39 → Medium Priority
- Age 40, Temperature 37 → Normal Priority

The AI preserved the original behavior. The conditional logic was simplified by combining the two Medium Priority cases into a single OR condition.

TASK 4 | IMPROVE DATA STRUCTURE USAGE

Task 4 – Improve Data Structure Usage

1. Original Code

```
patient_ids = [101, 205, 309, 412, 518, 623]

target_id = 412

found = False

for patient_id in patient_ids:
    if patient_id == target_id:
        found = True
        break

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

2. AI Prompt

Look at this Python code and suggest ways to improve its readability. Do not change the code yet. Explain what can be improved.

For this patient ID lookup code, would a set or a dictionary be better if patient IDs are searched frequently? Compare the two.

3. AI Suggestion

Copilot suggested improving readability by using a clearer variable name such as `is_patient_present`, reducing the manual loop.

For frequent patient ID lookups, Copilot recommended a set because the task only requires checking whether an ID exists. A set provides $O(1)$ lookup time.

Therefore, a set was selected for this use case.

4. Refactored Code

```
patient_ids = {101, 205, 309, 412, 518, 623}

target_id = 412

is_patient_present = target_id in patient_ids

if is_patient_present:
    print("Patient Found")
else:
    print("Patient Not Found")
```

5. Test Output

```
Existing patient ID:
target_id = 412
Output: Patient Found

Missing patient ID:
target_id = 999
Output: Patient Not Found
```

6. Explanation

The refactored code is more readable because the variable `is_patient_present` clearly describes what the Boolean value represents.

A set was chosen because the program only needs to perform membership checks. A list typically requires $O(n)$ time for membership checks.

The refactored code was tested with both an existing patient ID (412) and a missing patient ID (999), and both produced the expected output.

TASK 5 | REFACTOR AND IMPROVE CODE

Task 5 – Refactor and Improve Code

1. Original Code

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0

for student in students:
    total += student[1]

average = total / len(students)

print("Average:", average)

for student in students:
    if student[1] >= 50:
        print(student[0], "Pass")
    else:
        print(student[0], "Fail")
```

2. AI Prompt

Review this Python code and suggest at least 5 improvements for readability, maintainability, and Python style. Do not modify the code yet. For each suggestion, explain why it would improve the code. Include suggestions about meaningful variable names, reusable functions, and suitable Python features.

3. AI Suggestions

Copilot suggested the following improvements:

1. Use clearer, domain-specific variable names such as `student_scores`, `total_marks`, `pass_mark`, and `average_score`.
2. Replace the magic number 50 with a named constant such as `PASS_MARK = 50`.
3. Extract repeated or separate logic into reusable functions such as `calculate_average()` and `classify_students()`.
4. Separate data processing from output.
5. Use Python features such as type hints and structured data.
6. Avoid indexing into tuples and use tuple unpacking such as `for name, score in students`.
7. Use suitable built-in Python tools such as `statistics.mean()` where appropriate.
8. Encapsulate the script in a `main()` function.
9. Keep logic short and single-purpose.
10. Add docstrings or comments where needed.

4. Accepted and Rejected Suggestions

1. Clearer variable names — ACCEPTED Reason: Improves readability and makes the purpose of variables clearer. **2. Named constant `PASS_MARK = 50` — ACCEPTED Reason:** Removes the magic number and makes the passing rule explicit. **3. Reusable functions — ACCEPTED Reason:** Separates responsibilities and makes the logic easier to reuse and maintain. **4. Separate data processing from output — REJECTED Reason:** This would add unnecessary complexity for this small program. **5. Dataclass / structured data — REJECTED Reason:** A simple list of tuples is sufficient for this assignment. **6. Tuple unpacking — ACCEPTED Reason:** Makes the student name and score easier to understand than numeric indexes. **7. `statistics.mean()` — REJECTED Reason:** The existing average calculation is simple and clear. **8. `main()` function — REJECTED Reason:** It is unnecessary for this small assignment. **9. Single-purpose logic — ACCEPTED Reason:** Dedicated functions make each part of the program easier to understand. **10. Additional docstrings/comments — REJECTED Reason:** The current logic is simple enough that extra documentation is not necessary.

5. Refactored Code

```
student_scores = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

PASS_MARK = 50

def calculate_average(scores):
    total_marks = 0
    for name, score in scores:
        total_marks += score
    return total_marks / len(scores)

def print_pass_fail_status(scores, pass_mark):
    for name, score in scores:
        if score >= pass_mark:
            print(name, "Pass")
```

```
        else:
            print(name, "Fail")

average_score = calculate_average(student_scores)

print("Average:", average_score)

print_pass_fail_status(student_scores, PASS_MARK)
```

6. Test Output

Original version:

Average: 65.75

Arun Pass

Priya Fail

Rahul Pass

Meena Pass

Refactored version:

Average: 65.75

Arun Pass

Priya Fail

Rahul Pass

Meena Pass

The outputs are exactly the same, confirming that the refactored code preserves the original behavior.

7. Explanation

The refactoring improved readability and maintainability by using meaningful variable names, replacing the magic number 50 with `PASS_MARK`.

The most useful improvement was creating reusable functions. The functions `calculate_average()` and `print_pass_fail_status()` encapsulate logic that is used multiple times.

The refactored program was run and produced exactly the same output as the original program, so the original behavior was preserved.